set a value with list[5] = "five". The Elixir language doesn't provide a convenient syntax to access lists using index positions. Some functions let you access a list by index, but they aren't often used.

One pragmatic example of this is that you'll rarely see if length(list) == 0. The length/1 function is O(n), and there's a better way to make this comparison. Instead, use if list == [] to check if a list is empty. This is quick and doesn't involve iterating over the list. This applies to any data structure—Elixir is very efficient at checking data equality.

Keyword Lists

There's a special type of list that has its own syntax. A keyword list consists of tuple-pair elements in a list where the first element is an atom. This definition is a little complex, so it's best to see it in action by entering this code into IEx:

```
$ iex
iex> [hello: 1]
[hello: 1]

iex> [hello: 1, another: "value"] == [{:hello, 1}, {:another, "value"}]
true
```

Elixir expresses keyword lists with shorthand syntax of [atom: value, atom: value]. They are most commonly used to pass options into a function. For example, create lib/examples/modules/keyword_list.ex and enter the following code:

elixir_examples/lib/examples/modules/keyword_list.ex

```
defmodule Examples.Modules.KeywordList do
  def maybe_print(arg, opts) do
    print? = Keyword.get(opts, :print?, true)

    if print? do
      IO.puts("printing #{arg}")
    end

    arg
  end
end
```

Then run it in IEx:

```
$ iex -S mix
iex> alias Examples.Modules.KeywordList
iex> KeywordList.maybe_print(1, [])
printing 1
1

iex> KeywordList.maybe_print(1, print?: false)
1
```